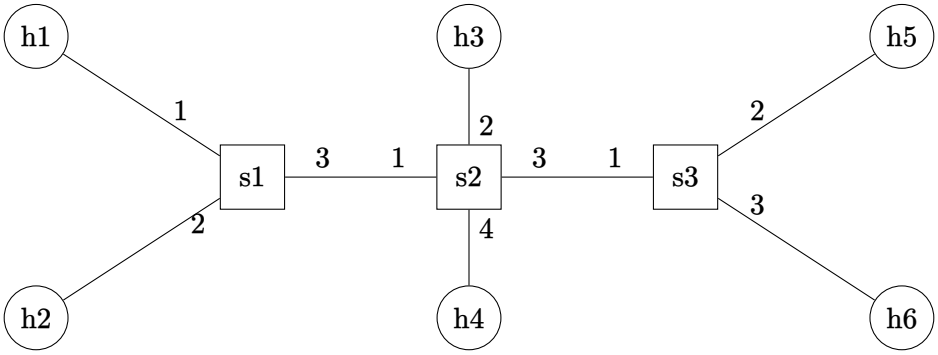
Software Defined Networking
10 febbraio 2024

Esercizio 1

Si consideri la rete in figura. I nodi h1,..., hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., hn sono MAC1,..., MACn e IP1,..., IPn. Sui nodi h1,..., hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse. Il controllore Ryu è collegato ai nodi s1,..., sm. La funzione di gestione dell'evento PacketIn è data nel Programma 1. Le interfacce tra gli switch e il controllore non sono indicate in figura. Se non altrimenti specificato, il TTL iniziale è 200.



Negli switch sono preinstallate le regole in Tabella 1.

L'host h1 porta 3322 invia un datagramma UDP verso h4 porta 123.

1. Scrivere se $s1$ invia messaggi al controllore. In caso affermativo scrivere quale tipo di messaggio openflow arriva in risposta dal controllore.

   Soluzione: no

2. Scrivere se $s2$ invia messaggi al controllore. In caso affermativo scrivere quale tipo di messaggio openflow arriva in risposta dal controllore.

   Soluzione: sì, una flow-mod e una packet-out

3. Scrivere se $s3$ invia messaggi al controllore. In caso affermativo scrivere quale tipo di messaggio openflow arriva in risposta dal controllore.

   Soluzione: no

L'host h2 porta 333 invia un datagramma UDP verso h4 porta 123.

4. Scrivere tutti gli host della rete che ricevono il pacchetto. Per ogni host scrivere il TTL.

Soluzione: h1 con TTL= 200 e h4 con TTL=199

L'host h1 porta 333 apre una connessione TCP verso h4 porta 123.

5. Scrivere se $s2$ invia messaggi al controllore. In caso affermativo scrivere quale tipo di messaggio openflow arriva in risposta dal controllore.

Soluzione: sì, packet-out

6. Quale regola si può inserire in s2 per fare in modo che il pacchetto di risposta sia ~~inviato ad~~ ricevuto in h1 con TTL=199?

Soluzione:

```
priority = 20
match = [ eth_type=IPv4, ip_dst=IP1, ip_src=IP4,
      ip_proto=TCP, tcp_src=123, tcp_dst=333 ]
action=[ DecTTL(), Output(1) ]
```

Tabella 1: Regole iniziali

Switch s1		
Priority	Match	Action
5	*	Output(FLOOD)
10	eth_type = IPv4 ip_proto=TCP	Output(CONTROLLER)
15	eth_dst = MAC2	DecTTL, Output(2)
15	eth_dst = MAC1	DecTTL, Output(1)

Switch s2		
Priority	Match	Action
0	*	Output(FLOOD)
10	eth_type = IPv4	Output(CONTROLLER)

Switch s3		
Priority	Match	Action
0	*	Output(FLOOD)
15	eth_dst = MAC5	DecTTL, Output(2)
15	eth_dst = MAC6	DecTTL, Output(3)

Programma 1 – Packet In

```
1
2  @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
3  def packet_in_handler(self, ev):
4     msg = ev.msg; datapath = msg.datapath; ofproto = datapath.ofproto;
5     parser = datapath.ofproto_parser; in_port = msg.match['in_port']
6     pkt = packet.Packet(msg.data); pkt_eth = pkt.get_protocol(eth.eth)
7     pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
8     pkt_udp = pkt.get_protocol(udp.udp); pkt_tcp = pkt.get_protocol(tcp.tcp)
9     data = msg.data
10
11    if (datapath.id == 2 and pkt_udp is not None and pkt_ipv4.dst == IP4):
12       match = parser.OFPMatch(eth_type=ETH_TYPE_IP,
13          ipv4_dst = pkt_ipv4.dst, ip_proto=IPPROTO_UDP,
14          udp_dst=pkt_udp.dst_port)
15       actions = [
16          parser.OFPDecTTL(),
17          parser.OFPActionOutput(4) ]
18       inst = [ parser.OFPInstructionActions(
19          ofproto.OFPIT_APPLY_ACTIONS,actions)]
20       mod = parser.OFPFlowMod(datapath=datapath, priority=20,
21          match=match, instructions=inst)
22       datapath.send_msg(mod)
23
24    actions = [parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
25    out = parser.OFPPacketOut(datapath=datapath,
26             buffer_id=ofproto.OFP_NO_BUFFER,data=msg.data,
27             in_port=in_port, actions=actions)
28    datapath.send_msg(out)
```

Esercizio 2

Si consideri un'interfaccia gestita da uno scheduler Deficit Round Robin. La velocità di trasmissione è 8 Gbit/s. Al tempo $t = 0$ sono presenti i seguenti pacchetti, di cui viene indicata la lunghezza in byte, ordinati dal più recente al più vecchio:

Coda 1:	50, 50, 150
Coda 2:	200, 50, 50
Coda 3:	100, 150, 100
Coda 4:	50

Il quanto vale 100 per tutte le code. Al tempo $t = 250$ ns si presenta alla coda 4 un pacchetto di 150 byte.

7. Scrivere quanti pacchetti trasmette ogni coda al primo round.

   Soluzione: 0, 2, 1, 1

8. Scrivere il deficit di tutte le code alla fine del primo round.

   Soluzione: 100, 0, 0, 0

9. Scrivere quanti pacchetti trasmette ogni coda al secondo round.

   Soluzione: 2, 0, 0, 0

10. Scrivere il tempo totale impiegato per trasmettere tutti i pacchetti.

    Soluzione: 1 100 ns

Si consideri ora uno scenario in cui tutte le code hanno un quanto di 200.

11. Scrivere, nel nuovo scenario, quanti pacchetti trasmette ogni coda al primo round.

    Soluzione: 2, 2, 1, 2

12. Scrivere il tempo totale impiegato per trasmettere tutti i pacchetti.

    Soluzione: 1 100 ns

Esercizio 3

Si consideri il seguente set di regole, per il quale implementare un meccanismo di Bit Vector Linear Search:

Source Address	Destination Address	Source Port	Destination Port	Protocol
80.0.0.0/5	*	80	*	TCP
80.0.0.0/4	128.0.0.0/2	80	22	TCP
*	96.0.0.0/3	*	53	UDP
88.0.0.0/5	*	*	123	UDP
92.0.0.0/6	64.0.0.0/2	23	*	UDP
92.0.0.0/7	64.0.0.0/3	25	80	*

13. Scrivere il bitvector per la entry 88.0.0.0/5 del campo Source Address.

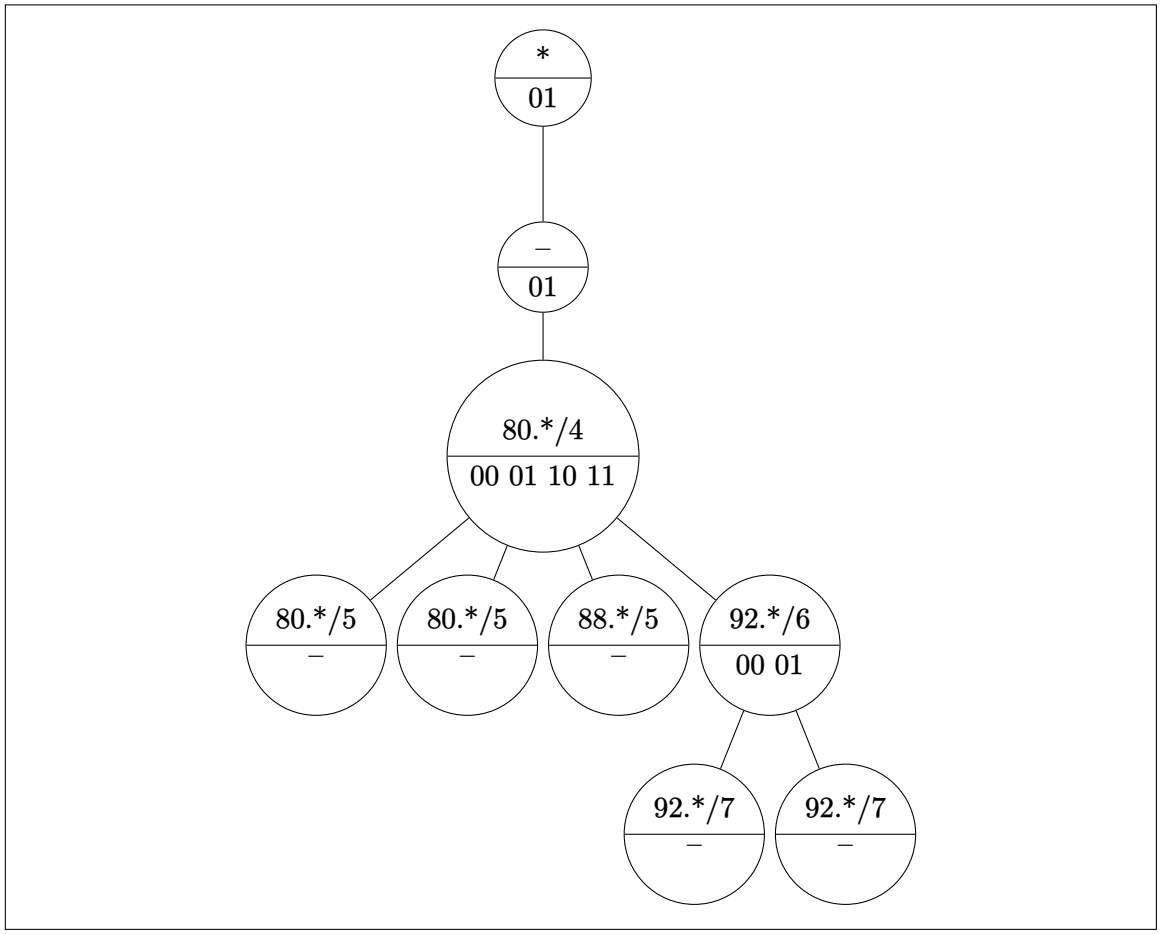
Soluzione: 011100

14. Scrivere il bitvector per la entry 92.0.0.0/6 del campo Source Address.

Soluzione: 011110

15. La ricerca degli indirizzi IP è implementata con un algoritmo multibit trie a passo 2. Disegnare il trie per il campo Source Address.

Soluzione:



16. La ricerca esatta è implementata con una hash table in cui la funzione hash è $h(x) = x \mod 10$. Disegnare la tabella hash per il campo Destination Port.

Soluzione:

